rxnet — Guía de usuario: C

FSM y Redes de Petri sobre un runtime síncrono reactivo

2026-05-20

Contents

rxnet — Guía de usuario **5**

1. El problema que resuelve rxnet 5
2. La hipótesis síncrona 5
3. El tick y sus fases 6
    Por qué esa separación de fases 6
4. Máquinas de estado finito (FSM) 7
    4.1 Cuándo usar una FSM 7
    4.2 Anatomía de una máquina 7
    4.3 Guards y acciones 8
    4.4 Callbacks de fase 8
    4.5 Ejemplo completo: luz con auto-apagado 9
    4.6 El patrón de señales de entrada en FSM 10
5. Redes de Petri (PN) 11
    5.1 Cuándo usar una red de Petri 11
    5.2 Conceptos fundamentales 11
    5.3 Semántica greedy-sequential 12
    5.4 Anatomía de una red 12
    5.5 Lugares de señal (signal places) 13
    5.6 Ejemplo completo: luz toggle 14
6. Modelos de ejecución concurrente 15
    6.1 Ejecutivo cíclico — `rx_cyclic_exec` 15
    6.2 Multitarea cooperativa — `rx_coop_exec` 16
    6.3 Threads paralelos — `rx_thread_exec` 16
    6.4 Resumen comparativo de executors 18
    6.5 Comunicación entre nodos 18
7. Cuándo usar FSM, cuándo PN 19
    Usa FSM cuando… 19
    Usa PN cuando… 19
    Mezclar ambos (patrón habitual) 20

8. Referencia rápida de la API 20
Core runtime 20
FSM 21
Petri Net 22
Firmas de callbacks 23
9. Depuración y trazado 24
Activar en tiempo de compilación 24
Uso básico 24
Trazado de fases 25
Eventos de usuario 26
Visualizar desde el Mac de desarrollo 26
Hooks de plataforma 26
10. Extracción de diagramas y verificación formal 26
Instalación 27
Uso desde la línea de comandos 27
Target `diagrams` del Makefile 27
Cómo funciona 27
Ejemplo de salida DOT 28
Lectura adicional 28

List of Figures

rxnet — Guía de usuario

1. El problema que resuelve rxnet

Los sistemas reactivos responden a eventos del entorno: pulsaciones de botón, lecturas de sensor, mensajes de red, expiración de temporizadores. El reto no es *qué hacer* ante un evento, sino *cuándo hacerlo* y *qué hay que ver* en ese momento.

Sin disciplina, el código reactivo acumula dos tipos de bugs difíciles de reproducir:

- **Race conditions de lectura**: el guard de una transición lee un bit de hardware a mitad de ciclo, cuando otro módulo ya lo consumió.
- **Efectos secundarios prematuros**: una acción ejecutada durante la evaluación modifica estado que otro módulo todavía no evaluó.

rxnet impone una disciplina que los elimina por construcción: **toda la información que entra en el sistema se lee exactamente una vez por ciclo, al principio, y todos los efectos secundarios se ejecutan exactamente una vez, al final**.

---

2. La hipótesis síncrona

rxnet está inspirada en los *lenguajes síncronos* (Esterel, Lustre, Signal) y en sus derivados industriales (SCADE, Simulink). La idea central se llama la **hipótesis síncrona**:

> La computación de un tick es instantánea. El mundo externo no cambia mientras el sistema está procesando.

Esto es una abstracción, no una afirmación física. En la práctica significa:

- El tick debe completarse antes de que llegue el siguiente evento significativo.
- Si el tick tarda más que el período de muestreo, el sistema tiene un problema de diseño, no de concurrencia.

Bajo la hipótesis síncrona, **no hay carrera de datos posible dentro del tick**: todos los módulos leen el mismo snapshot de entradas, y nadie ve los cambios de los demás hasta el siguiente tick.

3. El tick y sus fases

Cada llamada a `rx_tick()` ejecuta cuatro fases de nodo en orden estricto:

#	Fase	Qué hace cada nodo
1	**Latch**	`latch_inputs_cb(ctx, user)` — leer GPIO, calcular señales derivadas, tomar snapshot
2	**Evaluate**	`evaluate()` — decidir siguiente estado / flags (solo lectura del snapshot)
3	**Commit**	`commit()` — aplicar decisiones; tras el commit de todos, ejecutar acciones diferidas
4	**Dump**	`dump_outputs_cb(ctx, user)` — escribir GPIO, actualizar salidas

Por qué esa separación de fases

La separación **evaluate / commit / dump** no es arbitraria:

- **Evaluate** solo puede leer, nunca escribir estado observable. Todos los módulos ven el mismo estado del sistema al mismo tiempo.
- **Commit** aplica las decisiones. Las acciones diferidas encoladas durante `evaluate` se ejecutan después de que todos los nodos activos hayan hecho commit, antes de cualquier `dump`. Una acción ve el estado comprometido de todos los módulos, no solo el propio.
- **Dump** publica las salidas después de commit y de las acciones diferidas.

La separación **latch / dump** garantiza que las lecturas de hardware ocurren *antes* de evaluar (snapshot limpio) y las escrituras *después* de decidir (salida consistente).

---

4. Máquinas de estado finito (FSM)

4.1 Cuándo usar una FSM

Una FSM es la herramienta adecuada cuando el comportamiento del módulo depende de su **historia**: no solo del evento actual sino de lo que pasó antes.

Ejemplos típicos:

Problema	Estados naturales
Luz con botón	OFF, ON
Semáforo	ROJO, VERDE, AMARILLO
Puerta con cerrojo	ABIERTA, CERRADA, BLOQUEADA
Protocolo de comunicación	IDLE, CONECTANDO, CONECTADO, ERROR
Menú de display	MENU_PRINCIPAL, SUBMENU_A, SUBMENU_B

4.2 Anatomía de una máquina

Una `rx_fsm_machine` se define completamente en tiempo de compilación: transiciones, estados y callbacks son punteros a funciones estáticas. No hay allocación dinámica.

```c
#include "rxnet/fsm.h"

/* Estados: enteros arbitrarios */
enum { STATE_OFF = 0, STATE_ON = 1 };

/* Datos de usuario pasados a todos los callbacks */
typedef struct {
    bool latched_event;
    int  output_enabled;
} light_data_t;

/* Tabla de transiciones (puede ser static const) */
static const rx_fsm_transition transitions[] = {
    /* { from_state, to_state, guard, action } */
    { STATE_OFF, STATE_ON,  button_pressed, light_on  },
    { STATE_ON,  STATE_OFF, button_pressed, light_off },
};
```

```
18
19 static light_data_t data;
20 static rx_fsm_machine machine;
21
22 rx_fsm_machine_init(
23     &machine,
24     "light",                                /* nombre (debug) */
25     STATE_OFF,                              /* estado inicial */
26     transitions,
27     sizeof(transitions) / sizeof(*transitions),
28     &data,                                  /* user pointer */
29     light_latch_cb,                         /* fase latch */
30     light_dump_cb                           /* fase dump */
31 );
```

Semántica first-match: en cada tick, el runtime recorre las transiciones en orden de declaración y ejecuta la *primera* cuya guardia devuelva distinto de cero y cuyo estado origen coincida con el estado actual.

4.3 Guards y acciones

```
1  /* Guard: pura, sin efectos secundarios; devuelve != 0 para "verdadero"
       */
2  static int button_pressed(const rx_fsm_context *ctx, void *user) {
3      const light_data_t *d = user;
4      (void)ctx;
5      return d->latched_event;
6  }
7
8  /* Acción: deferred, corre tras commit con estado ya comprometido */
9  static void light_on(rx_fsm_context *ctx, void *user) {
10     light_data_t *d = user;
11     (void)ctx;
12     d->output_enabled = 1;
13 }
14
15 static void light_off(rx_fsm_context *ctx, void *user) {
16     light_data_t *d = user;
17     (void)ctx;
18     d->output_enabled = 0;
19 }
```

Regla importante: los guards nunca deben tener efectos secundarios. Son funciones puras de lectura. Los efectos van en las acciones.

4.4 Callbacks de fase

```c
/* Fase latch: tomar snapshot de entradas */
static void light_latch_cb(rx_fsm_context *ctx, void *user) {
    light_data_t *d = user;
    (void)ctx;
    d->latched_event = read_button_gpio();
}

/* Fase dump: escribir salidas */
static void light_dump_cb(rx_fsm_context *ctx, void *user) {
    const light_data_t *d = user;
    (void)ctx;
    set_led_gpio(d->output_enabled);
}
```

4.5 Ejemplo completo: luz con auto-apagado

```c
#include <stdint.h>
#include <stdbool.h>
#include "rxnet/fsm.h"

enum { STATE_OFF = 0, STATE_ON = 1 };

typedef struct {
    bool     latched_event;
    bool     enabled;
    uint32_t timeout_ms;
    uint32_t deadline_ms;
    uint32_t now_ms;
} auto_light_data_t;

static int pressed(const rx_fsm_context *ctx, void *user) {
    (void)ctx;
    return ((auto_light_data_t *)user)->latched_event;
}

static int timed_out(const rx_fsm_context *ctx, void *user) {
    const auto_light_data_t *d = user;
    (void)ctx;
    return d->now_ms >= d->deadline_ms;
}

static void turn_on(rx_fsm_context *ctx, void *user) {
    auto_light_data_t *d = user;
    (void)ctx;
    d->enabled = true;
    d->deadline_ms = d->now_ms + d->timeout_ms;
}

```

```c
static void turn_off(rx_fsm_context *ctx, void *user) {
    auto_light_data_t *d = user;
    (void)ctx;
    d->enabled = false;
}

static void latch_cb(rx_fsm_context *ctx, void *user) {
    auto_light_data_t *d = user;
    (void)ctx;
    d->now_ms = get_time_ms();
    d->latched_event = read_button_gpio();
}

static void dump_cb(rx_fsm_context *ctx, void *user) {
    const auto_light_data_t *d = user;
    (void)ctx;
    set_led_gpio(d->enabled);
}

static const rx_fsm_transition transitions[] = {
    { STATE_OFF, STATE_ON,  pressed,   turn_on  },
    { STATE_ON,  STATE_ON,  pressed,   turn_on  },  /* reset timer */
    { STATE_ON,  STATE_OFF, timed_out, turn_off },
};

/* Montaje */
static auto_light_data_t data = { .timeout_ms = 5000 };
static rx_fsm_machine    machine;

void light_init(void) {
    rx_fsm_machine_init(
        &machine, "auto_light", STATE_OFF,
        transitions, sizeof(transitions) / sizeof(*transitions),
        &data, latch_cb, dump_cb
    );
}
```

4.6 El patrón de señales de entrada en FSM

La forma canónica de conectar hardware a una FSM en rxnet:

Hardware	`latch_inputs_cb`	guards / actions
GPIO → evento	`data->latched = read_gpio()`	`if (data->latched)...`
(vivo, volátil)	(snapshot estable)	(solo leen snapshot)

El callback `latch_inputs_cb` es el único punto de contacto con el hardware. El resto de la máquina trabaja con copias estables.

---

5. Redes de Petri (PN)

5.1 Cuándo usar una red de Petri

Una red de Petri es la herramienta adecuada cuando hay **concurrencia natural**: varios procesos que avanzan en paralelo y necesitan sincronizarse.

Ejemplos típicos:

Problema	Modela bien porque…
Botón toggle	Un token fluye entre P_OFF y P_ON
Buffer productor-consumidor	Tokens representan slots llenos/vacíos
Semáforo / mutex	Un token en P_LIBRE controla el acceso
Protocolo de handshake	Ambos lados avanzan y se sincronizan
Recursos compartidos	Tokens cuentan instancias disponibles

La PN no reemplaza a la FSM: cuando el comportamiento es esencialmente lineal (un agente con historia), la FSM es más clara. Cuando hay múltiples flujos que se sincronizan, la PN es más clara.

5.2 Conceptos fundamentales

Concepto	Símbolo	Significado
Lugar (place)	○	condición, estado, recurso, mensaje pendiente
Token	•	unidad de recurso, condición activa
Transición	rect.	evento, paso de proceso

Concepto	Símbolo	Significado
Arco de entrada	→T	condición necesaria para disparar (consume tokens)
Arco de salida	T→	condición que produce al disparar (produce tokens)

Una transición **está habilitada** cuando todos sus lugares de entrada tienen suficientes tokens (≥ peso del arco). Una transición habilitada **dispara**: consume los tokens de sus entradas y produce tokens en sus salidas.

5.3 Semántica greedy-sequential

rxnet usa evaluación **greedy-sequential**: las transiciones se evalúan en orden de declaración, y las anteriores *consumen tokens inmediatamente*, antes de que las posteriores se evalúen.

Consecuencia práctica: **el orden de declaración implica prioridad**. Si dos transiciones compiten por el mismo token, gana la que se declara primero.

```
static const rx_pn_arc consume_x[] = {{ .place_id = P_X, .weight = 1 }};
static const rx_pn_arc produce_a[] = {{ .place_id = P_A, .weight = 1 }};
static const rx_pn_arc produce_b[] = {{ .place_id = P_B, .weight = 1 }};

static const rx_pn_transition transitions[] = {
    /* T0 declarada antes → prioridad sobre T1 */
    { consume_x, 1, produce_a, 1, NULL, NULL },
    /* T1: no dispara en el mismo tick si T0 consumió el token de P_X */
    { consume_x, 1, produce_b, 1, NULL, NULL },
};
```

5.4 Anatomía de una red

```
#include "rxnet/pn.h"

enum { P_OFF = 0, P_ON = 1, P_REQUEST = 2 };

static const rx_pn_arc arcs_off_req[]  = {{ P_OFF, 1 }, { P_REQUEST, 1 }};
```

```c
static const rx_pn_arc arcs_on[]       = {{ P_ON,  1 }};
static const rx_pn_arc arcs_on_req[]   = {{ P_ON,  1 }, { P_REQUEST, 1 }};
static const rx_pn_arc arcs_off[]      = {{ P_OFF, 1 }};

static const rx_pn_transition transitions[] = {
    /* off + request → on */
    { arcs_off_req, 2, arcs_on,  1, NULL, NULL },
    /* on  + request → off */
    { arcs_on_req,  2, arcs_off, 1, NULL, NULL },
};

static const int initial_places[] = { 1, 0, 0 };  /* token en P_OFF */
static rx_pn_net net;

rx_pn_net_init(
    &net,
    "light",
    initial_places, 3,
    transitions,    2,
    NULL,           /* user pointer */
    light_latch_cb, /* fase latch — NULL → noop */
    light_dump_cb   /* fase dump — NULL → noop */
);
```

Los arrays de arcos y transiciones pueden ser **`static const`**: el runtime no los modifica, solo los lee. Los tokens (`net.places[]`) sí se modifican en cada tick.

5.5 Lugares de señal (signal places)

Un **lugar de señal** no acumula tokens: se *restablece* en cada latch a 0 o 1 según una condición del entorno.

```c
/* En latch_inputs_cb: reescribir P_TOGGLE_DUE en cada tick */
static void blink_latch_cb(rx_pn_context *ctx, void *user) {
    rx_pn_net *net = user;
    bool is_blinking = net->places[P_X1] > 0 || net->places[P_X2] > 0;
    net->places[P_TOGGLE_DUE] = (is_blinking && timer_elapsed()) ? 1 : 0;
    (void)ctx;
}
```

Contrasta con los **lugares de cola** (como P_REQUEST), que acumulan tokens:

```c
/* En latch_inputs_cb: añadir un token si hay pulsación */
static void light_latch_cb(rx_pn_context *ctx, void *user) {
    rx_pn_net *net = user;
    if (read_button_gpio())
```

```
        net->places[P_REQUEST]++;   /* acumula; se consume 1 por tick
           */
    (void)ctx;
}
```

Tipo de lugar	Comportamiento	Ejemplo
Señal	Se reescribe a 0/1 cada latch	Timer expirado, sensor activo
Cola	Acumula; transición consume 1	Pulsación de botón pendiente
Estado	Token único que fluye	ON/OFF, ROJO/VERDE
Contador	N tokens = N recursos	Slots libres en buffer

5.6 Ejemplo completo: luz toggle

```c
#include <stdbool.h>
#include "rxnet/pn.h"

enum { P_OFF = 0, P_ON = 1, P_REQUEST = 2 };

typedef struct {
    bool output;
} light_pn_data_t;

static void action_turn_on(rx_pn_context *ctx, void *user) {
    ((light_pn_data_t *)user)->output = true;  (void)ctx;
}

static void action_turn_off(rx_pn_context *ctx, void *user) {
    ((light_pn_data_t *)user)->output = false;  (void)ctx;
}

static void latch_cb(rx_pn_context *ctx, void *user) {
    rx_pn_net *net = user;
    if (read_button_gpio())
        net->places[P_REQUEST]++;
    (void)ctx;
}

static void dump_cb(rx_pn_context *ctx, void *user) {
    set_led_gpio(((light_pn_data_t *)user)->output);  (void)ctx;
}

static const rx_pn_arc arcs_off_req[] = {{ P_OFF, 1 }, { P_REQUEST, 1
   }};
static const rx_pn_arc arcs_on[]      = {{ P_ON,  1 }};
```

```c
static const rx_pn_arc arcs_on_req[]  = {{ P_ON,  1 }, { P_REQUEST, 1 }};
static const rx_pn_arc arcs_off[]     = {{ P_OFF, 1 }};

static const rx_pn_transition transitions[] = {
    { arcs_off_req, 2, arcs_on,  1, NULL, action_turn_on  },
    { arcs_on_req,  2, arcs_off, 1, NULL, action_turn_off },
};

static const int      initial[]  = { 1, 0, 0 };
static light_pn_data_t pn_data;
static rx_pn_net       net;

void light_pn_init(void) {
    pn_data.output = false;
    rx_pn_net_init(
        &net, "light",
        initial, 3,
        transitions, 2,
        &net,      /* user = net para acceder a places[] en latch_cb */
        latch_cb, dump_cb
    );
}
```

---

6. Modelos de ejecución concurrente

Esta sección explica cómo rxnet se relaciona con los tres modelos clásicos de ejecución en sistemas embebidos y de tiempo real.

rxnet incluye tres **executors** que gestionan el timing y el scheduling automáticamente. El período de cada runtime se lee de `rt->period_us`, que el propio runtime calcula como el MCD de los períodos de sus nodos. Antes de ejecutar `latch`, el executor escribe en el `rx_context` el instante lógico de activación del grupo; las callbacks pueden consultarlo con `rx_context_activation_us(ctx)`. Todos los nodos de un mismo grupo de activación ven el mismo valor, también en `rx_thread_exec`.

6.1 Ejecutivo cíclico — `rx_cyclic_exec`

Tabla de despacho estática con hiperperíodo. Un único hilo, orden de slots determinista. Adecuado para bare-metal y configuraciones RTOS simples.

```c
#include "rxnet/cyclic.h"

rx_cyclic_exec ce;
rx_cyclic_exec_init(&ce);
rx_cyclic_exec_add(&ce, &fast_rt.runtime);   /* período 10 ms */
rx_cyclic_exec_add(&ce, &slow_rt.runtime);   /* período 20 ms */
rx_cyclic_exec_run(&ce);  /* retorna tras rx_cyclic_exec_stop(&ce) */
```

El executor calcula automáticamente: - `base = MCD(10, 20)= 10 ms` → período base - `hyper = MCM(10, 20)= 20 ms` → 2 slots - Slot 0: `fast_rt` + `slow_rt`; Slot 1: solo `fast_rt`

Cuándo usarlo: periodos fijos conocidos en compilación, determinismo máximo.

Limitaciones: si los períodos no son armónicos, el hiperperíodo puede crecer mucho; en ese caso suele convenir `rx_coop_exec` o `rx_thread_exec`.

6.2 Multitarea cooperativa — `rx_coop_exec`

Scheduling dinámico por deadline. Un único hilo comprueba qué nodos están vencidos en cada runtime, ordena los nodos activos por deadline efectivo y ejecuta sólo ese grupo. No materializa una tabla de hiperperíodo.

```c
#include "rxnet/coop.h"

rx_coop_exec ce;
rx_coop_exec_init(&ce);
rx_coop_exec_add(&ce, &rt_a.runtime);   /* período 10 ms */
rx_coop_exec_add(&ce, &rt_b.runtime);   /* período 15 ms */
rx_coop_exec_run(&ce);  /* retorna tras rx_coop_exec_stop(&ce) */
```

El executor avanza cada activación desde su último disparo (no desde “ahora”), evitando la acumulación de deriva de fase incluso cuando un nodo se retrasa.

Cuándo usarlo: períodos irregulares, tareas que a veces se alargan un poco, sin overhead de threads.

Limitaciones: una tarea muy lenta puede retrasar a todas las demás.

6.3 Threads paralelos — `rx_thread_exec`

Un thread por nodo periódico. El executor crea grupos de activación para cada instante absoluto activo, sin materializar una tabla de hiperperíodo. Dos barreras BSP por grupo sincronizan las fases reactivas con verdadero paralelismo:

- `eval_b`: todos los nodos activos han hecho `latch` y `evaluate`.
- `commit_b`: todos los nodos activos han hecho `commit` y despachado acciones diferidas.

El último nodo del último runtime corre en el hilo llamante (útil para el nodo CLI/stdin). Los nodos async (`period_us = 0`) no están soportados en `rx_thread_exec`, porque un solo thread async no puede participar con seguridad en varios grupos de activación solapados.

```c
#include "rxnet/thread.h"

rx_thread_exec te;
rx_thread_exec_init(&te);
rx_thread_exec_add(&te, &pn_rt.runtime);   /* nodos PN → threads */
rx_thread_exec_add(&te, &cli_rt.runtime);  /* CLI → hilo principal */
rx_thread_exec_run(&te);  /* retorna tras rx_thread_exec_stop(&te) */
```

Todos los executors ofrecen un hook `rx_*_exec_on_stop()` para ejecutar una operación de cierre de la aplicación/modelo una vez antes de que `run()` retorne.

Cuándo usarlo: varios nodos con trabajo de cómputo intensivo que se benefician del paralelismo real; sistemas con múltiples cores.

Limitaciones: overhead de sincronización de barreras; requiere `-lpthread` en POSIX (o el soporte de tasks equivalente en FreeRTOS/Zephyr).

El análisis automático de planificabilidad se activa con `rx_*_exec_enable_sched_check(&exec, 1)` y puede reportarse con `rx_*_exec_check_schedulability(&exec, &report, log)`. `rx_cyclic_exec` comprueba que el hiperperíodo construido con los WCETs medidos cabe en los plazos; `rx_coop_exec` aplica análisis iterativo de tiempo de respuesta con el bloqueo máximo de tareas menos prioritarias; `rx_thread_exec` usa análisis de tiempo de respuesta sólo cuando se ha podido configurar FIFO de prioridades fijas. Si FIFO no está disponible, el executor puede seguir funcionando, pero el análisis se reporta como no soportado.

Las secciones críticas sobre recursos compartidos se delimitan con `rx_context_critical_begin(ctx, resource_id)` y `rx_context_critical_end(ctx)`. En `rx_thread_exec` el contexto usa un `rx_mutex_t` por recurso; el puerto POSIX intenta inicializarlo con `PTHREAD_PRIO_INHERIT`, y los puertos RTOS usan el mutex nativo. En `rx_cyclic_exec` y `rx_coop_exec` no se toma ningún mutex porque la sección ya queda contenida en el bloque atómico del tick, pero se mide igualmente. El runtime conserva el máximo observado de cada par tarea/recurso y lo incluye en `rx_sched_report`.

```c
static void machine_eval(rx_node *node, rx_context *ctx)
{
    rx_context_critical_begin(ctx, 1);
    shared_state.value++;
    rx_context_critical_end(ctx);
```

```
6 }
```

En cada tarea del informe, `blocking_us` es el bloqueo máximo, `interference_us` la interferencia máxima y `response_us` el tiempo de respuesta peor caso. Para `rx_thread_exec`, el bloqueo se calcula como la suma máxima de combinaciones de secciones críticas de tareas de menor prioridad, sin elegir dos secciones de la misma tarea ni dos secciones del mismo recurso. La interferencia se calcula iterativamente: primero se asume una activación de cada tarea de mayor prioridad en el instante crítico, y después se recalcula `I = sum(ceil(R/Tj)* Cj)` con el último `R = C + B + I` hasta que el tiempo de respuesta converge o supera el plazo.

6.4 Resumen comparativo de executors

	`rx_cyclic_exec`	`rx_coop_exec`	`rx_thread_exec`
Threads	1	1	1 por nodo periódico
Dispatch	Tabla estática (hiperperíodo del executive)	Próxima activación + deadline	Grupos de activación + barreras BSP
Períodos	Mejor con períodos armónicos o hiperperíodo corto	Cualquiera	Cualquiera
Paralelismo	No	No	Sí
Overhead	Mínimo	Mínimo	Barreras mutex
Casos típicos	Bare-metal, RTOS simple	Períodos irregulares	Múltiples cores

6.5 Comunicación entre nodos

Estado de FSM como señal: un nodo puede leer `machine_a.state` directamente en sus guards (el estado fue comprometido en commit, antes de que cualquier nodo ejecute el siguiente tick).

Tokens de PN como mensajes: el productor escribe `net.places[P_BUFFER]` en su `latch_inputs_cb`; el consumidor dispara la transición que consume ese token.

Acciones diferidas: cualquier callback puede encolar una acción para el despacho deferred del tick actual:

```c
rx_context_enqueue_deferred_action(ctx, my_action_fn, user_ptr);
```

Las acciones se ordenan por prioridad (FIFO dentro del mismo nivel):

```c
rx_context_enqueue_deferred_action_p(ctx, send_alarm,   user,
    RX_PRIORITY_CRITICAL);
rx_context_enqueue_deferred_action_p(ctx, log_event,    user,
    RX_PRIORITY_NORMAL);
rx_context_enqueue_deferred_action_p(ctx, update_stats, user,
    RX_PRIORITY_LOW);
```

Nota sobre thread safety: `rx_tick()` no es thread-safe por sí mismo. Con `rx_thread_exec`, las barreras garantizan la consistencia del snapshot; con los otros executors (hilo único) no se necesita sincronización adicional.

---

7. Cuándo usar FSM, cuándo PN

Usa FSM cuando...

- El módulo tiene **un agente con historia**: hay un único "estado del módulo" en cada momento.
- Las transiciones son mutuamente excluyentes: estar en un estado excluye estar en otro.
- El control de flujo es lineal (incluso si tiene bucles y ramas).
- Necesitas guards complejos que leen múltiples variables.

Ejemplos:

- Semáforo: ROJO → VERDE → AMARILLO → ROJO
- Cerrojo: DESBLOQUEADO → BLOQUEADO
- Protocolo: IDLE → SYN_SENT → ESTABLISHED

Usa PN cuando...

- Hay **múltiples agentes independientes** que se sincronizan.
- Los recursos son contables (N slots, M conexiones disponibles).
- El paralelismo es natural: varias actividades ocurren al mismo tiempo.
- Necesitas modelar producción/consumo de forma explícita.

Ejemplos:

- Productor/consumidor: places `P_LLENOS` + `P_VACIOS`
- Semáforo binario: `P_MUTEX` (0 o 1 token)
- Rendezvous: A espera a B, B espera a A

Mezclar ambos (patrón habitual)

Lo más potente es combinarlos en un único runtime base. La FSM describe la **política** (qué modo estoy); la PN describe los **recursos y sincronización** (qué tengo disponible):

```c
/* Sistema de acceso a sala:
 * FSM: gestiona el ciclo de apertura de puerta
 * PN:  gestiona los recursos (tokens = personas dentro) */

rx_context      ctx;
rx_runtime      rt;
rx_fsm_machine  door_machine;
rx_pn_net       capacity_net;

rx_context_init(&ctx);
rx_runtime_init(&rt, &ctx, 2);

door_fsm_init(&door_machine);
capacity_pn_init(&capacity_net);

rx_runtime_add_node(&rt, &door_machine.node, 0, 0);
rx_runtime_add_node(&rt, &capacity_net.node, 0, 0);

for (;;) {
    rx_tick(&rt);   /* FSM y PN avanzan juntos en el mismo tick */
    sleep_10ms();
}
```

Al compartir el mismo `rx_context`, las acciones diferidas de la FSM y de la PN se ejecutan todas en el mismo despacho deferred del tick, después de commit y antes de dump.

---

8. Referencia rápida de la API

Core runtime

```c
#include "rxnet/runtime.h"

/* Contexto: cola de acciones diferidas */
```

```c
rx_context ctx;
rx_context_init(&ctx);

/* Runtime: lista de nodos */
rx_runtime rt;
rx_runtime_init(&rt, &ctx, node_capacity);

/* Registrar nodos */
rx_runtime_add_node(&rt, &machine.node, period_us, deadline_us);   /* FSM o PN */

/* Ejecutar un ciclo */
rx_tick(&rt);  /* tick manual: ejecuta todos los nodos */

/* Instante lógico del grupo activo (lo fijan los executors antes de latch) */
long activation_us = rx_context_activation_us(&ctx);

/* Encolar acción diferida con prioridad por defecto (NORMAL) */
rx_context_enqueue_deferred_action(&ctx, action_fn, user_ptr);

/* Encolar acción diferida con prioridad explícita */
rx_context_enqueue_deferred_action_p(&ctx, action_fn, user_ptr, RX_PRIORITY_HIGH);

/* Prioridades disponibles */
/* RX_PRIORITY_CRITICAL = 3   alarmas, paradas de emergencia */
/* RX_PRIORITY_HIGH     = 2   control en tiempo real          */
/* RX_PRIORITY_NORMAL   = 1   lógica de aplicación (defecto)  */
/* RX_PRIORITY_LOW      = 0   telemetría, logging             */

/* Worker pool asíncrono (implementación provista por el usuario) */
rx_runtime_set_worker_pool(&rt, &my_pool.base);  /* activar */
rx_runtime_set_worker_pool(&rt, NULL);            /* volver a modo síncrono */

/* Vtable que el usuario debe implementar */
struct rx_worker_pool {
    void (*post)(rx_worker_pool *self,
                 rx_deferred_action_fn fn,
                 rx_context *ctx,
                 void *user,
                 rx_priority_t priority);
};
```

FSM

```c
#include "rxnet/fsm.h"

```

```c
/* Tipos de función */
typedef int  (*rx_fsm_guard_fn)(const rx_fsm_context *ctx, void *user);
typedef void (*rx_fsm_action_fn)(rx_fsm_context *ctx, void *user);
typedef void (*rx_fsm_node_phase_fn)(rx_fsm_context *ctx, void *user);

/* Transición */
struct rx_fsm_transition {
    int                from_state;
    int                to_state;
    rx_fsm_guard_fn    guard;   /* NULL → siempre dispara */
    rx_fsm_action_fn   action;  /* NULL → sin efecto */
};

/* Inicializar máquina (stack allocation) */
void rx_fsm_machine_init(
    rx_fsm_machine          *machine,
    const char              *name,
    int                      initial_state,
    const rx_fsm_transition *transitions,
    size_t                   transition_count,
    void                    *user,
    rx_fsm_node_phase_fn     latch_inputs,   /* NULL → noop */
    rx_fsm_node_phase_fn     dump_outputs    /* NULL → noop */
);

/* Runtime dedicado FSM (contiene contexto propio) */
rx_fsm_runtime fsm_rt;
rx_fsm_runtime_init(&fsm_rt, machine_capacity);
rx_fsm_runtime_add_machine(&fsm_rt, &machine, period_us, deadline_us);
rx_fsm_tick(&fsm_rt);
```

Petri Net

```c
#include "rxnet/pn.h"

/* Tipos de función */
typedef int  (*rx_pn_guard_fn)(const rx_pn_context *ctx, void *user);
typedef void (*rx_pn_action_fn)(rx_pn_context *ctx, void *user);
typedef void (*rx_pn_node_phase_fn)(rx_pn_context *ctx, void *user);

/* Arco */
struct rx_pn_arc {
    size_t place_id;
    int    weight;
};

/* Transición */
struct rx_pn_transition {
    const rx_pn_arc *consume;       /* array de arcos de entrada */
```

```
    size_t           consume_count;
    const rx_pn_arc *produce;       /* array de arcos de salida */
    size_t           produce_count;
    rx_pn_guard_fn   guard;         /* NULL → siempre dispara si
       habilitada */
    rx_pn_action_fn  action;        /* deferred, NULL → noop */
};

/* Inicializar red (stack allocation) */
int rx_pn_net_init(
    rx_pn_net               *net,
    const char              *name,
    const int               *initial_places,
    size_t                   place_count,
    const rx_pn_transition  *transitions,
    size_t                   transition_count,
    void                    *user,
    rx_pn_node_phase_fn      latch_inputs,   /* NULL → noop */
    rx_pn_node_phase_fn      dump_outputs    /* NULL → noop */
);

/* Runtime dedicado PN */
rx_pn_runtime pn_rt;
rx_pn_runtime_init(&pn_rt, net_capacity);
rx_pn_runtime_add_net(&pn_rt, &net, period_us, deadline_us);
rx_pn_tick(&pn_rt);
```

Firmas de callbacks

```
/* Guard: pura, sin efectos secundarios; devuelve != 0 para verdadero
   */
static int my_guard(const rx_fsm_context *ctx, void *user) {
    (void)ctx;
    return ((my_data_t *)user)->latched_event;
}

/* Acción: deferred, corre tras commit con estado ya comprometido */
static void my_action(rx_fsm_context *ctx, void *user) {
    (void)ctx;
    ((my_data_t *)user)->output = 1;
}

/* Latch: snapshot hardware, señales derivadas */
static void latch_cb(rx_fsm_context *ctx, void *user) {
    (void)ctx;
    my_data_t *d = user;
    d->latched_event = read_gpio(d->button_pin);
}

```

```c
/* Dump: escribir salidas al hardware */
static void dump_cb(rx_fsm_context *ctx, void *user) {
    (void)ctx;
    const my_data_t *d = user;
    write_gpio(d->led_pin, d->output);
}
```

---

9. Depuración y trazado

rxnet incluye un subsistema de trazado **completamente opcional**: cuando no se activa, genera **cero código y cero datos**. Todos los macros se expanden a `((void)0)`, los campos extra de los nodos no existen, y el enlazador no incluye ningún símbolo relacionado.

Activar en tiempo de compilación

```
CFLAGS += -DRX_TRACE_ENABLE
```

Con este flag, `rx_node` gana dos campos (`trace` y `trace_nid`), y el runtime registra eventos `NODE_START`/`END`, transiciones FSM y disparos PN en un buffer circular de capacidad fija (sin heap).

Uso básico

```c
#include "rxnet/trace.h"

rx_trace_buf_t tracer;

/* 1. Inicializar el buffer (phases=0: sin eventos de fase; =1: con
   ellos) */
rx_trace_init(&tracer, 0);

/* 2. Adjuntar a cada nodo que quieras trazar */
rx_trace_attach(&tracer, &light_machine.node, 0);   /* nid = 0 */
rx_trace_attach(&tracer, &blink_machine.node, 1);   /* nid = 1 */

/* 3. Registrar nombres (opcional, para el informe HTML) */
rx_trace_set_node_name (&tracer, 0, "light");
rx_trace_set_state_name(&tracer, 0, STATE_OFF, "OFF");
rx_trace_set_state_name(&tracer, 0, STATE_ON,  "ON");

```

```c
rx_trace_set_node_name (&tracer, 1, "blink");
rx_trace_set_state_name(&tracer, 1, BLINK_SLOW, "SLOW");
rx_trace_set_state_name(&tracer, 1, BLINK_FAST, "FAST");

/* 4. Ejecutar el sistema normalmente... */

/* 5. Exportar a fichero binario */
rx_trace_export(&tracer, "trace.rxnt");
```

`rx_trace_attach()` sigue siendo útil cuando quieres controlar manualmente el `nid` de cada nodo. Para la mayoría de los casos, sin embargo, ahora existe una opción más cómoda a nivel de runtime:

```c
rx_trace_init(&tracer, 0);

/* Adjunta todos los nodos registrados en rt.runtime */
rx_trace_attach_runtime(&tracer, &rt.runtime);
```

`rx_trace_attach_runtime()` es **idempotente e incremental**: si se invoca dos veces sobre el mismo runtime no renumera ni vuelve a adjuntar los nodos ya conocidos, y si entre llamadas se añaden nuevos nodos al runtime, una nueva invocación asigna `trace_nid` solo a esos nodos nuevos.

Ejemplo:

```c
rx_trace_attach_runtime(&tracer, &rt.runtime);

/* más tarde: se amplía la red */
rx_fsm_runtime_add_machine(&rt, &aux_machine, 0, 0);
rx_runtime_build(&rt.runtime);

rx_trace_attach_runtime(&tracer, &rt.runtime);  /* solo adjunta
   aux_machine */
```

Para redes de Petri, registrar también lugares y transiciones:

```c
rx_trace_set_node_name (&tracer, 2, "blink_pn");
rx_trace_set_place_name(&tracer, 2, P_OFF,    "OFF");
rx_trace_set_place_name(&tracer, 2, P_ON,     "ON");
rx_trace_set_trans_name(&tracer, 2, T_TOGGLE, "toggle");
```

Trazado de fases

Pasar `phases=1` a `rx_trace_init` registra el inicio y fin de cada fase (latch / eval / commit / dump), permitiendo medir cuánto tarda cada una:

```c
rx_trace_init(&tracer, 1);  /* phases activadas */
```

Eventos de usuario

Desde cualquier parte del código:

```c
/* lid = label id (0..RX_TRACE_MAX_LABELS-1); value = uint16_t */
rx_trace_user(&tracer, 0, 42);

/* Registrar el nombre del label */
rx_trace_set_label_name(&tracer, 0, "temperatura");
```

Visualizar desde el Mac de desarrollo

Una vez que tienes el fichero `.rxnt`, ejecuta el decodificador Python:

```
python -m rxnet.tools.trace trace.rxnt --report trace.html --open
python -m rxnet.tools.trace trace.rxnt --stats          # WCRT por nodo
    en texto
python -m rxnet.tools.trace trace.rxnt --perfetto out.json
```

El informe HTML contiene diagramas de cada FSM/PN y una tabla WCRT. El fichero Perfetto se puede abrir en ui.perfetto.dev para ver la línea de tiempo interactiva.

Hooks de plataforma

Por defecto el trazado usa el reloj y el mutex de `rxnet/port.h` (POSIX, FreeRTOS o Zephyr según la plataforma destino). Se pueden sobreescribir antes de incluir `trace.h`:

```c
#define RX_TRACE_NOW_NS()          my_platform_clock_ns()
#define RX_TRACE_LOCK_TYPE         my_lock_t
#define RX_TRACE_LOCK_INIT(lk)     my_lock_init(&(lk))
#define RX_TRACE_LOCK_ACQUIRE(lk)  my_lock_acquire(&(lk))
#define RX_TRACE_LOCK_RELEASE(lk)  my_lock_release(&(lk))
#include "rxnet/trace.h"
```

---

10. Extracción de diagramas y verificación formal

La herramienta `rxnet-extract` lee directamente el código C y genera diagramas de estado o una especificación TLA+ sin ningún paso intermedio. El código fuente C es la única fuente de verdad — no hace falta mantener YAML ni ningún fichero de anotaciones adicional.

Instalación

```
pip install rxnet          # instala rxnet-extract en $PATH
# o bien:
brew install rxnet-python
```

Requiere `clang` en `PATH` (cualquier versión de Apple Clang o LLVM Clang).

Uso desde la línea de comandos

```
# Graphviz DOT → SVG / PDF
rxnet-extract light_fsm.c -Iinclude --dot | dot -Tsvg -o light.svg
rxnet-extract light_fsm.c -Iinclude --dot | dot -Tpdf -o light.pdf

# Mermaid stateDiagram-v2 (se renderiza en GitHub Markdown)
rxnet-extract light_fsm.c -Iinclude --mermaid

# TLA+ para model checking con TLC
rxnet-extract light_fsm.c -Iinclude --tla > light_fsm.tla

# Varios formatos en un solo paso (separados por '---')
rxnet-extract light_fsm.c -Iinclude --tla --dot --mermaid

# Los argumentos desconocidos se pasan a clang
rxnet-extract light_fsm.c -Iinclude -DDEBUG=1 --dot
```

Target `diagrams` del Makefile

El `Makefile` de la biblioteca C genera diagramas para todos los ejemplos FSM:

```
cd c
make diagrams   # produce .svg, .pdf y .tla para cada ejemplo
```

Los ficheros se escriben junto al código fuente de cada ejemplo.

Cómo funciona

`rxnet-extract` ejecuta `clang -Xclang -ast-dump=json -fsyntax-only` sobre el fichero C y analiza el AST JSON resultante sin ninguna dependencia adicional:

1. Busca `EnumConstantDecl` con `_STATE_` en el nombre → tabla de estados.
2. Busca el `VarDecl` cuyo `qualType` contiene `rx_fsm_transition[` → tabla de transiciones.

3. Busca la `CallExpr` a `rx_fsm_machine_init` → nombre de la máquina y estado inicial.

Con esa información construye un `FsmSpec` interno y lo vuelca en el formato pedido.

Ejemplo de salida DOT

```
// FSM: light
digraph {
    rankdir=LR;
    node [shape=circle fontname="Helvetica" fontsize=11];
    __start [shape=point width=0.2];
    __start -> "OFF";
    "OFF" -> "ON" [label="button_pressed / light_on" fontsize=10];
    "ON" -> "OFF" [label="button_pressed / light_off" fontsize=10];
}
```

Lectura adicional

- **Código fuente**: `c/` — runtime (~200 líneas), fsm (~150 líneas), pn (~200 líneas)
- **Tests**: `c/tests/` — test_fsm.c, test_pn.c, parity_runner.c
- **Ejemplos**: `c/examples/fsm/` y `c/examples/pn/` — ejecutables para macOS/Linux
- **Especificaciones**: `docs/specs/c/requirements.md` y `docs/specs/c/design.md`
- **Implementación Python**: `python/` — misma semántica, API ergonómica en Python 3.14+